Rapport d'activité - FISE Groupe 10

- Anaé DESIGAUX
- Hugo DARTOY
- Noah ANDRIAMAMPIANINA
- Paul-Alexandre DEGUILHEN

Sommaire

- Grammaire
- Lexer
- Parser V1
- Parser V2
- Structure AST
- Affichage SDL AST
- Difficultés rencontrées
- Exemples
- Répartition du travail

Grammaire

Voir annexe Grammaire

Lexer

Le lexer est construit en deux parties : une partie qui reconnaît les opérateurs (+, - , &&,...) et une autre qui lit les chaînes de caractères et reconnaît les expressions réservées (if, else, class, for,...).

Parser V1

Version avec stack des NUT, en reprenant la table SD avec des actions sémantiques dans chacune des cases valides.
Problème rencontré : impossible de stocker le nom d'une variable ou d'une classe par exemple, ce qui rend la construction des AST impossible.

Parser V2

Ecrire une fonction de parsing par règle => fonctions imbriquées.
Squelette d'une fonction pour la règle classe :

```
classe -> public_opt static_opt "class" "ident_type" extends_opt "{" decl_list "}" .
parse_classe():
    pub = parse_public()
    stat = parse_static()
    match("class")
    class_name = match(id)
    extended_class = parse_extends_opt()
    match("{")
    declarations = parse_decl_list()
    match("}")
```

En cas de rencontre d'erreur syntaxique, le parser utilise la méthode panicmode :
le lexer avance jusqu'au prochain `;` ou `}` ou `{`, puis continue le parsing à cet endroit.

Structure AST

Voir annexe Structure AST

TypeChecker (Analyse sémantique)

Parcours de l'AST construit pour relever les erreurs sémantiques de type.

Affichage SDL AST

Affichage dans une fenêtre SDL de l'AST construit. Possibilité de naviguer dans l'arbre avec les flèches du clavier ou avec la souris et de zoomer/dézoomer avec la molette. Réinitialiser la vue avec la touche R.

Difficultés rencontrées

- Rendre la grammaire LL(1) : factorisations, récursivités gauches… conflits à gérer => timing serré
- Première implémentation du parser incompatible avec la construction des AST
- Panicmode : conflits ponctuels dans plusieurs fonctions parse_X() lorsque le panicmode

s'active. Le Lexeur avance par exemple à un ; alors que la prochaine ligne de la fonction parse_X() doit lire) .
Entre autres, il était complexe de trouver une méthode qui marche pour chacune des fonctions parse_X()

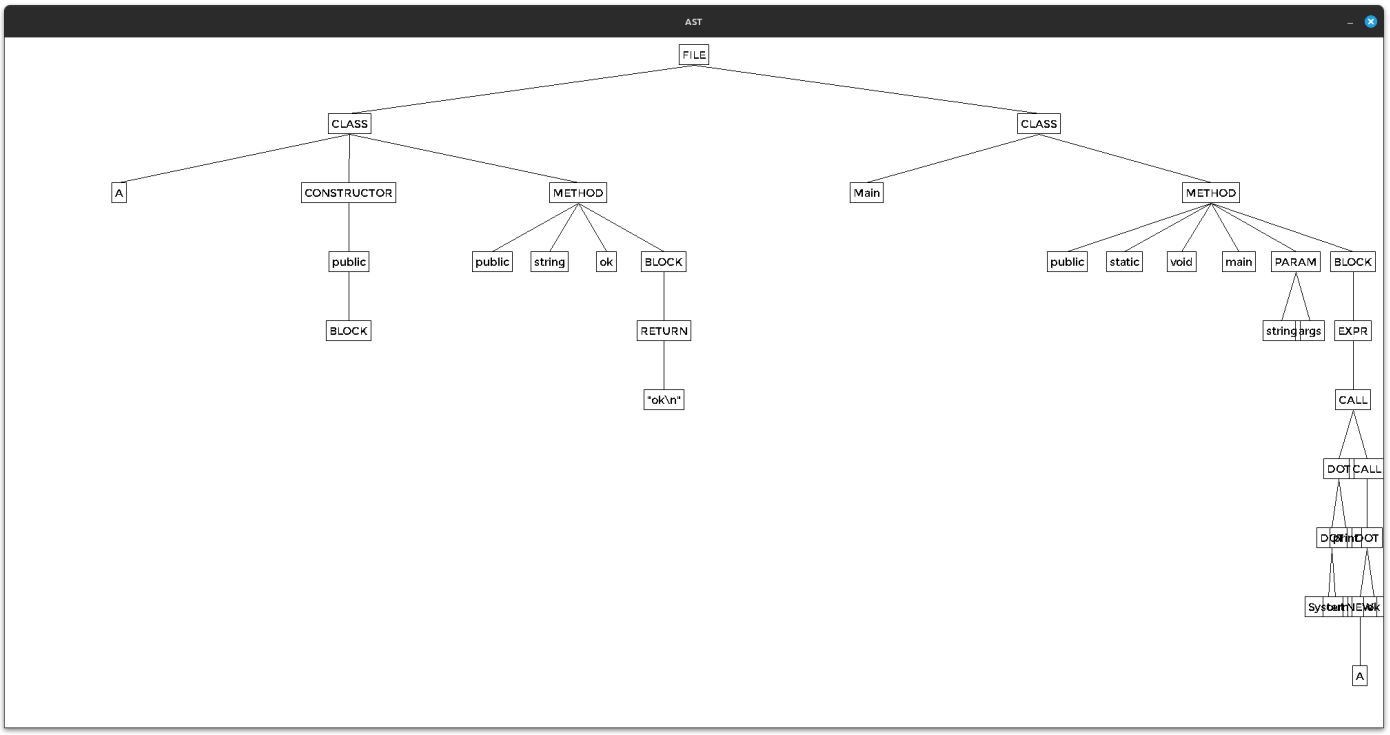
Exemples

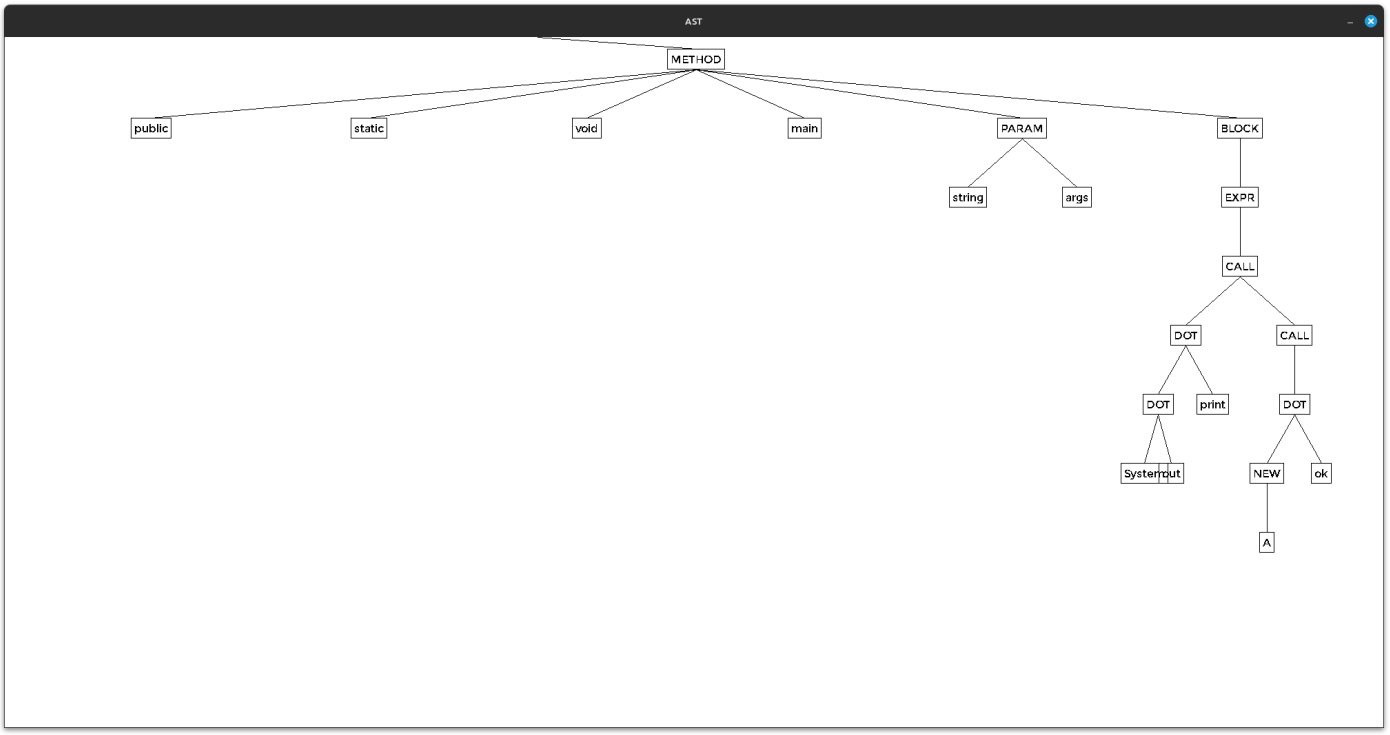
Exemple 1

Programme JURA

```
class A {
    A() {}
    String ok() { return "ok\n"; }
}
class Main {
    public static void main(String args[]) {
        System.out.print(new A().ok());
    }
}
```

AST affiché



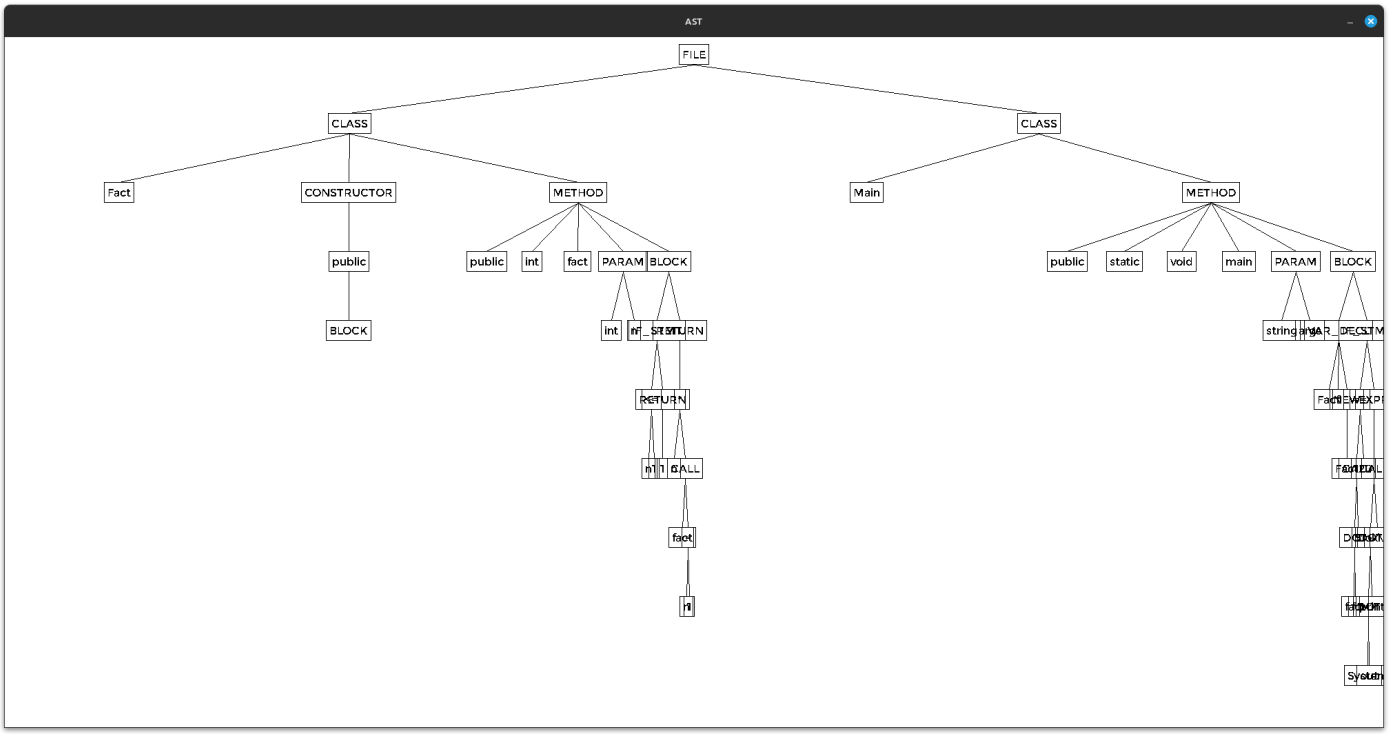


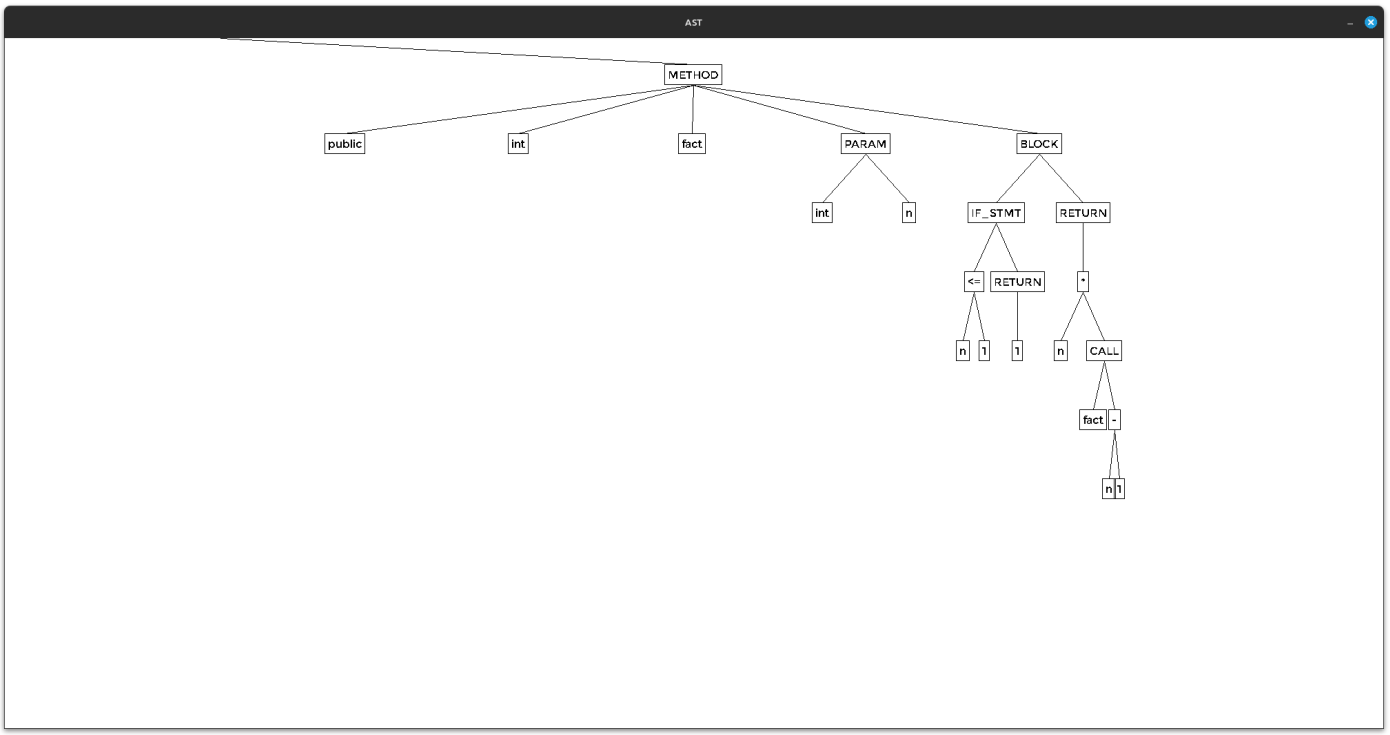
Exemple 2

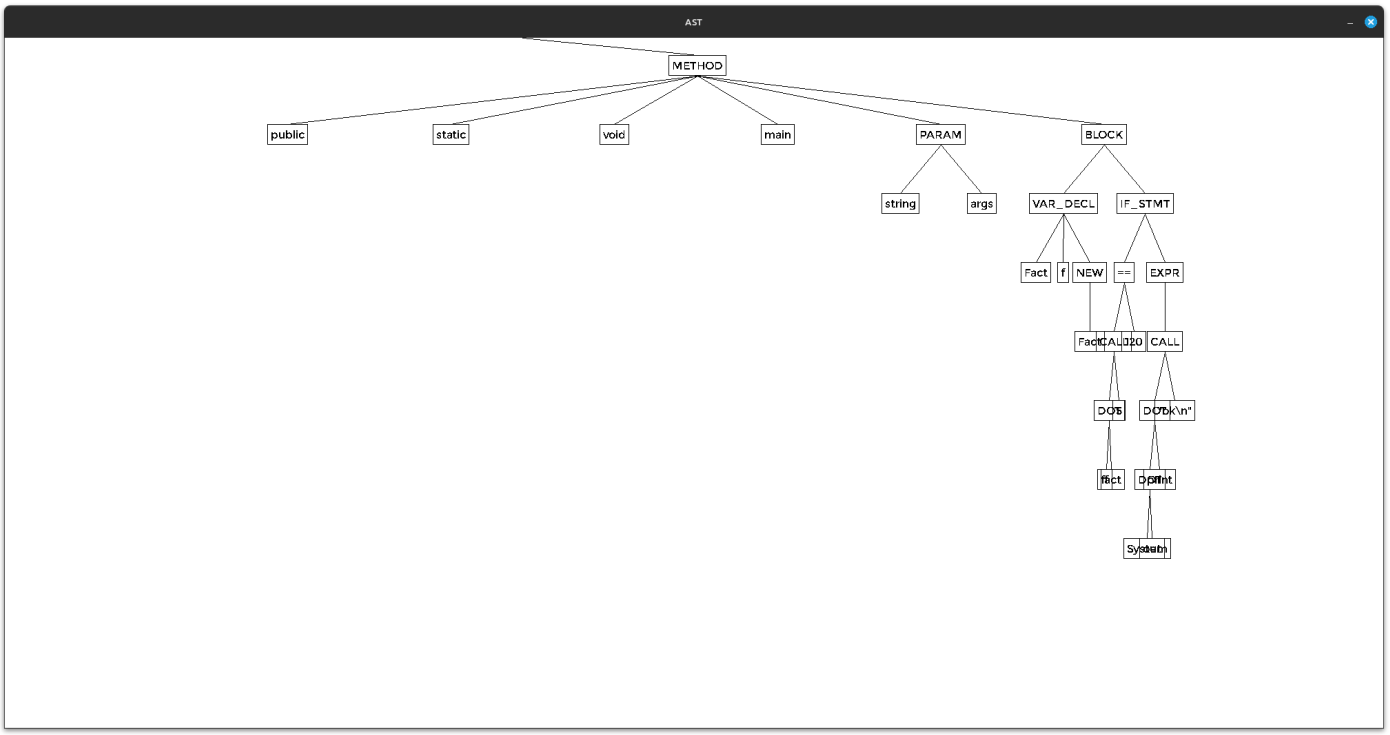
Programme JURA

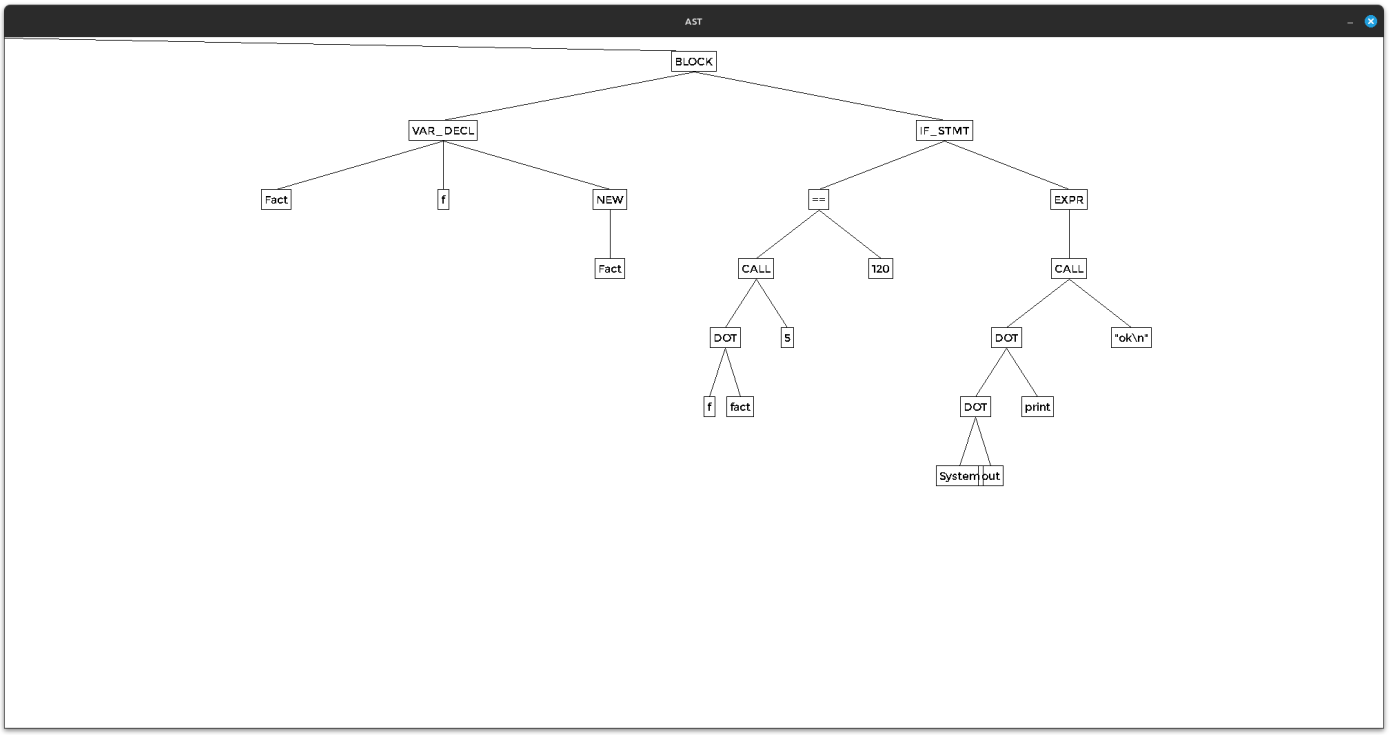
```
class Fact {
    Fact() {}
    int fact_rec(int n) {
        if (n <= 1) return 1;
        return n * fact_rec(n - 1);
    }
}
class Main {
    public static void main(String args[]) {
        Fact f = new Fact();
        if (f.fact_rec(5) == 120)
            System.out.print("ok\n");
    }
}
```

AST affiché


AST
FILE
CLASS
CLASS
Fact
CONSTRUCTOR
METHOD
Main
METHOD
public
public
int
fact
PARAM
BLOCK
public
static
void
main
PARAM
BLOCK
BLOCK
int
RETURN
fact


AST
METHOD
public
int
fact
PARAM
BLOCK
int
n
IF_STMT
RETURN
<=
RETURN
*
n
1
1
n
CALL
fact
-
n
1





Répartition du travail

Voir annexe Répartition du travail

Annexe **Grammaire**

fichier -> class_list

class_list -> classe class_list
class_list -> ε

classe -> public_opt static_opt "class" "ident_type" extends_opt "{" decl_list "}"

public_opt -> "public"
public_opt -> ε

static_opt -> "static"
static_opt -> ε

extends_opt -> "extends" "ident_type"
extends_opt -> ε

decl_list -> decl decl_list
decl_list -> ε

decl -> "public" decl_public
decl -> "static" const_or_meth
decl -> type const_or_meth_or_attr

decl_public -> const_or_meth
decl_public -> "static" const_or_meth

const_or_meth_or_attr -> "ident" meth_or_attr
const_or_meth_or_attr -> function_tail

meth_or_attr -> function_tail
meth_or_attr -> "=" expr ";"
meth_or_attr -> ";"

function_tail -> "(" params_opt ")" "{" stmt_list "}"

const_or_meth -> type const_or_meth_sol

const_or_meth_sol -> function_tail
const_or_meth_sol -> "ident" function_tail

```
params_opt -> params
params_opt -> ε

params -> type "ident" cro_opt params_tail

cro_opt -> "[" "]"
cro_opt -> ε

params_tail -> ε
params_tail -> "," params

type -> "ident_type"
type -> "boolean"
type -> "int"
type -> "void"

stmt_list -> stmt stmt_list
stmt_list -> ε

stmt -> type "ident" "=" expr ";"
stmt -> type "ident" ";"
stmt -> expr_opt ";"
stmt -> "if" "(" expr ")" stmt stmt_if
stmt -> "for" "(" expr_opt ";" expr_opt ";" expr_opt ")" stmt
stmt -> block
stmt -> "return" expr_opt ";"

stmt_if -> "else" stmt
stmt_if -> ε

block -> "{" stmt_list "}"

expr_opt -> expr
expr_opt -> ε

expr -> ass_expr

ass_expr -> lor_expr ass_expr_suite
ass_expr_suite -> "=" ass_expr
ass_expr_suite -> ε

lor_expr -> land_expr lor_expr_suite
lor_expr_suite -> "||" land_expr lor_expr_suite
```

lor_expr_suite -> ε

land_expr -> equality_expr land_expr_suite
land_expr_suite -> "&&" equality_expr land_expr_suite
land_expr_suite -> ε

equality_expr -> relational_expr equality_expr_suite
equality_expr_suite -> "==" relational_expr equality_expr_suite
equality_expr_suite -> "!=" relational_expr equality_expr_suite
equality_expr_suite -> ε

relational_expr -> add relational_expr_suite
relational_expr_suite -> "<" add relational_expr_suite
relational_expr_suite -> "<=" add relational_expr_suite
relational_expr_suite -> ">" add relational_expr_suite
relational_expr_suite -> ">=" add relational_expr_suite
relational_expr_suite -> "instanceof" type relational_expr_suite
relational_expr_suite -> ε

add -> mult_expr add_suite
add_suite -> "+" mult_expr add_suite
add_suite -> "-" mult_expr add_suite
add_suite -> ε

mult_expr -> unary mult_suite
mult_suite -> "*" unary mult_suite
mult_suite -> "/" unary mult_suite
mult_suite -> "%" unary mult_suite
mult_suite -> ε

unary -> "!" unary
unary -> "-" unary
unary -> "+" unary
unary -> cast_or_postfix

cast_or_postfix -> "(" cast_or_postfix_par
cast_or_postfix -> postfix_expr

cast_or_postfix_par -> type ")" cast_or_postfix
cast_or_postfix_par -> expr ")" postfix_tail

postfix_expr -> primary_with_prefix postfix_tail

postfix_tail -> "." "ident" postfix_tail

postfix_tail -> "(" lexpr_opt ")" postfix_tail
postfix_tail -> ε

primary_with_prefix -> "!" primary_with_prefix
primary_with_prefix -> "-" primary_with_prefix
primary_with_prefix -> "+" primary_with_prefix
primary_with_prefix -> primary

primary -> "integer"
primary -> "string"
primary -> "true"
primary -> "false"
primary -> "null"
primary -> "ident"
primary -> "new" "ident_type" "(" lexpr_opt ")"
primary -> "(" expr ")"

lexpr_opt -> lexpr
lexpr_opt -> ε
lexpr -> expr lexpr_expr
lexpr_expr -> ε
lexpr_expr -> "," lexpr

Annexe Structure AST

File : Racine

- ClassDecl (n)

ClassDecl

- Modifieurs (public, private)
- Nom (String)
- Nom de la superclasse (String) (optionnel)
- Field (n)
- Constructor (n)
- Method (n)

Field

- Modifieurs (public, private, static)
- Type (1)
- Nom (String)
- Expression (1) (optionnel)

Constructor

- Type (1)
- Parameter (n)
- Statement (1)

Method

- Modifieurs (public, private)
- Nom (String)
- Type (1)
- Parameter (n)

Parameter

- Type (1)
- Nom (String)

Type

- Nom du type (String)

Statement

Pas un vrai noeud mais une interface pour les types suivants :

- Expression
- ReturnStmt
- IfStmt
- ForStmt
- Block
- VarDecl

ReturnStmt

- Expression (1) (optionnel)

IfStmt

- Condition : Expression (1)
- Instruction du if : Statement (1)
- Instruction du else : Statement (1) (optionnel)

ForStmt

- Statement (1) (optionnel)
- Expression (1) (optionnel)
- Expression (1) (optionnel)
- Instruction du for : Statement (1)

Block

- Statement (n)

VarDecl

- Type (1)
- Nom (String)
- Expression (1) (optionnel)

Expression

Pas un vrai noeud mais une interface pour les types suivants :

- Literal
- Nom de variable (String)
- BinaryOp
- UnaryOp
- CallExpr
- InstanceOf
- NewExpr
- FieldAccess
- Cast

Literal

L'un des objets suivants :

- Entier (int)
- Booléen (boolean)
- Chaîne de caractères (String)

BinaryOp

- Opérateur (int)
- Expression (1)
- Expression (1)

UnaryOp

- Opérateur (int)
- Expression (1)

CallExpr

- Appellé : Expression (1)
- Arguments : Expression (n)

InstanceOf

- Expression (1)
- Type (1)

NewExpr

- Type (1)
- Arguments : Expression (n)

FieldAccess

- Expression (1)
- Nom du champ (String)

Cast

- Type (1)
- Expression (1)

Annexe Répartition du travail

Anaé Desigaux

- Lexer (reconnaissance des opérateurs)
- Structure des AST
- Table d'analyse LL(1) (abandonnée)
- Implémentation des fonctions sémantiques
- Type Checker

Environ 80 heures

Hugo Dartoy

- Premier travail sur la grammaire
- Construction automatique d'une table d'analyse à partir d'une grammaire (abandonné)
- Affichage graphique de l'AST (SDL)
- Docker et CI/CD (tests automatiques)

Environ 85 heures

Noah Andriamampianina

- Lexer (lecture des Tokens, fonctions d'API)
- Table d'erreurs syntaxiques (abandonné)
- Structures des AST
- Parser (nouvelle version : fonctions de parsing)

Environ 85 heures

Paul-Alexandre Deguilhen

- Structure du Lexer (règles API)
- Implémentation Table LL(1) (abandonné)
- Implémentation des fonctions sémantiques
- Refonte de l'AST
- Parser (nouvelle version : fonctions de parsing)
- Trouvé une ressource pour les tests

Environ 85 heures